



Section 9 — Loops & Iteration

1. Overview

JavaScript-এ traditional `for` loop ছাড়াও দুইটা খুব গুরুত্বপূর্ণ loop আছে:

```
for...of
```

এবং:

```
for...in
```

দেখতে প্রায় একই রকম হলেও এদের কাজ একদম আলাদা।

সবচেয়ে important rule:

`for...of` → *values* দেয়

`for...in` → *keys* দেয়

এই একটা line মাথায় রাখলে অনেক confusion দূর হয়ে যাবে।

2. Quick Comparison

Loop	কী দেয়?	মূল ব্যবহার
<code>for</code>	index/control	সাধারণ iteration
<code>for...of</code>	values	arrays, strings, Set, Map, iterables
<code>for...in</code>	keys	objects-এর keys

Example:

```
const numbers = [10, 20, 30];
```

for...of

```
for (const value of numbers) {  
  console.log(value);  
}
```

Output:

```
10  
20  
30
```

for...in

```
for (const key in numbers) {  
  console.log(key);  
}
```

Output:

```
0  
1  
2
```

So:

```
for...of → 10, 20, 30  
for...in → 0, 1, 2
```

3. **for...of** কী?

Syntax:

```
for (const value of iterable) {  
  // code  
}
```

Example:

```
const fruits = ["Apple", "Banana", "Mango"];

for (const fruit of fruits) {
  console.log(fruit);
}
```

Output:

```
Apple
Banana
Mango
```

এখানে `fruit` প্রতিবার array-এর **actual value** পায়।

4. Traditional `for` vs `for...of`

Traditional:

```
const fruits = ["Apple", "Banana", "Mango"];

for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

`for...of`:

```
for (const fruit of fruits) {
  console.log(fruit);
}
```

দুটোর output একই:

```
Apple
Banana
Mango
```

কিন্তু `for...of` array-এর index manually manage করতে হয় না।

5. `for...of` Array-এর Values দেয়

```
const numbers = [10, 20, 30];

for (const number of numbers) {
  console.log(number);
}
```

Output:

```
10
20
30
```

এখানে:

```
number = 10
number = 20
number = 30
```

6. Index দরকার হলে?

`for...of` directly index দেয় না।

তবে `entries()` ব্যবহার করতে পারো:

```
const fruits = ["Apple", "Banana", "Mango"];

for (const [index, fruit] of fruits.entries()) {
  console.log(index, fruit);
}
```

Output:

```
0 Apple
1 Banana
2 Mango
```

এখানে array destructuring ব্যবহার হয়েছে:

```
[index, fruit]
```

7. `for...of` String-এর উপর

String একটি iterable।

```
const name = "Shariar";

for (const character of name) {
  console.log(character);
}
```

Output:

```
S
h
a
r
i
a
r
```

8. কেন String-এ `for...of` Useful?

Traditional:

```
const name = "Shariar";

for (let i = 0; i < name.length; i++) {
  console.log(name[i]);
}
```

`for...of`:

```
for (const character of name) {  
  console.log(character);  
}
```

Second version বেশি readable।

9. Unicode এবং `for...of`

`for...of` string-এর ক্ষেত্রে Unicode code points অনুযায়ী iterate করতে পারে।

Example:

```
const text = "A😊B";  
  
for (const char of text) {  
  console.log(char);  
}
```

Output:

```
A  
😊  
B
```

এটা traditional indexing-এর তুলনায় useful হতে পারে, কারণ JavaScript string-এর `.length` UTF-16 code units count করে।

10. `for...of` Set-এর উপর

`Set` একটি iterable collection।

```
const numbers = new Set([10, 20, 30]);  
  
for (const number of numbers) {  
  console.log(number);  
}
```

Output:

```
10
 20
 30
```

11. Set-এর Duplicate Behavior

Set duplicate values রাখে না।

```
const numbers = new Set([10, 10, 20, 20, 30]);

for (const number of numbers) {
  console.log(number);
}
```

Output:

```
10
 20
 30
```

12. `for...of` Map-এর উপর

`Map` -ও iterable।

```
const users = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

for (const entry of users) {
  console.log(entry);
}
```

Output conceptually:

```
["name", "Shariar"]  
["age", 31]
```

কারণ Map-এর default iteration প্রতিটি entry হিসেবে `[key, value]` দেয়।

13. Map Destructuring with `for...of`

আরও clean:

```
const users = new Map([  
  ["name", "Shariar"],  
  ["age", 31]  
]);  
  
for (const [key, value] of users) {  
  console.log(key, value);  
}
```

Output:

```
name Shariar  
age 31
```

এখানে:

```
[key, value]
```

হলো array destructuring।

14. Map-এর Keys Only

Map-এর keys চাইলে:

```
const users = new Map([
  ["name", "Shariar"],
  ["age", 31]
]);

for (const key of users.keys()) {
  console.log(key);
}
```

Output:

```
name
age
```

15. Map-এর Values Only

```
for (const value of users.values()) {
  console.log(value);
}
```

Output:

```
Shariar
31
```

16. Map-এর Entries

```
for (const [key, value] of users.entries()) {
  console.log(key, value);
}
```

এটাই সাধারণত সবচেয়ে readable pattern।

17. Iterable কী?

এখন আসি important concept-এ:

```
Iterable
```

Iterable হলো এমন object/data structure যেটাকে JavaScript iteration protocol-এর মাধ্যমে একটার পর একটা value হিসেবে iterate করা যায়।

Common iterables:

```
Array  
String  
Set  
Map
```

এগুলোতে `for...of` ব্যবহার করা যায়।

18. Examples of Iterables

Array

```
for (const item of [1, 2, 3]) {  
  console.log(item);  
}
```

String

```
for (const char of "Hello") {  
  console.log(char);  
}
```

Set

```
for (const value of new Set([1, 2, 3])) {  
  console.log(value);  
}
```

Map

```
for (const [key, value] of new Map([
  ["a", 1],
  ["b", 2]
])) {
  console.log(key, value);
}
```

19. Plain Object কি Iterable?

সাধারণ object:

```
const user = {
  name: "Shariar",
  age: 31
};
```

এটার উপর সরাসরি:

```
for (const value of user) {
  console.log(value);
}
```

করলে `TypeError` হবে, কারণ plain object defaultভাবে iterable নয়।

Object-এর ক্ষেত্রে সাধারণত:

```
Object.keys()
Object.values()
Object.entries()
```

ব্যবহার করা হয়।

20. Object-এর উপর `for...of` করতে চাইলে

`Object.values()` :

```
const user = {
  name: "Shariar",
  age: 31
};

for (const value of Object.values(user)) {
  console.log(value);
}
```

Output:

```
Shariar
31
```

`Object.entries()` :

```
for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

Output:

```
name Shariar
age 31
```

এটা object iteration-এর খুব useful modern pattern।

21. `for...in` কী?

`for...in` object-এর **enumerable property keys** iterate করে।

Syntax:

```
for (const key in object) {
  // code
}
```

Example:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  city: "Rangpur"  
};  
  
for (const key in user) {  
  console.log(key);  
}
```

Output:

```
name  
age  
city
```

22. `for...in` দিয়ে Value Access

`for...in` key দেয়।

Value পেতে:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
for (const key in user) {  
  console.log(user[key]);  
}
```

Output:

```
Shariar  
31
```

23. Key + Value Together

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
for (const key in user) {  
  console.log(key, user[key]);  
}
```

Output:

```
name Shariar  
age 31
```

Mental model:

```
key  
  ↓  
user[key]  
  ↓  
value
```

24. Why Bracket Notation?

কারণ `key` একটি variable।

✗ Wrong:

```
user.key
```

এটা `"key"` নামের property খুঁজবে।

Correct:

```
user[key]
```

এটা variable-এর actual value ব্যবহার করে property access করবে।

25. `for...in` with Arrays

Technically array-এর উপরও `for...in` কাজ করতে পারে:

```
const numbers = [10, 20, 30];

for (const index in numbers) {
  console.log(index);
}
```

Output:

```
0
1
2
```

তাই beginners মাঝে মাঝে মনে করে:

"Array-এর জন্য `for...in` use করা উচিত।"

কিন্তু সাধারণ array value iteration-এর জন্য এটা recommended না।

26. কেন Array-এর জন্য `for...in` Avoid করি?

কারণ `for...in` মূলত **property keys** iterate করে।

Array হলো object-এর একটি special form এবং array-তে custom enumerable properties-ও থাকতে পারে।

Example:

```
const numbers = [10, 20, 30];

numbers.extra = "Hello";

for (const key in numbers) {
  console.log(key);
}
```

এখানে output-এ numeric indexes-এর পাশাপাশি:

extra

আসতে পারে।

এটা array values iterate করার সময় unwanted behavior হতে পারে।

27. Array-এর জন্য Better Choice

যদি values দরকার:

```
for (const number of numbers) {  
  console.log(number);  
}
```

অথবা traditional:

```
for (let i = 0; i < numbers.length; i++) {  
  console.log(numbers[i]);  
}
```

So:

Array values → `for...of`

28. `for...in` মূলত Object-এর জন্য

Example:

```
const product = {  
  title: "Laptop",  
  price: 85000,  
  brand: "Dell"  
};  
  
for (const key in product) {  
  console.log(key);  
}
```

Output:

```
title
  price
  brand
```

29. `for...in` এবং Inherited Properties

এখানে একটি important advanced concept আছে।

JavaScript object prototype chain থেকে properties inherit করতে পারে।

`for...in` সাধারণভাবে enumerable inherited properties-ও দেখতে পারে।

Example:

```
const parent = {
  role: "admin"
};

const user = Object.create(parent);

user.name = "Shariar";

for (const key in user) {
  console.log(key);
}
```

Output-এর মধ্যে:

```
name
  role
```

থাকতে পারে।

কারণ `role` inherited property।

30. Own Property vs Inherited Property

Object-এর নিজস্ব property:

Own property

Prototype থেকে আসা property:

Inherited property

Example:

```
const parent = {  
  role: "admin"  
};  
  
const user = Object.create(parent);  
  
user.name = "Shariar";
```

এখানে:

user.name

→ own property

আর:

user.role

→ inherited property

31. `Object.hasOwnProperty()`

নিজের property কিনা check করতে modern approach:

```
Object.hasOwnProperty(object, key)
```

Example:

```
const parent = {
  role: "admin"
};

const user = Object.create(parent);

user.name = "Shariar";

console.log(Object.hasOwn(user, "name"));
console.log(Object.hasOwn(user, "role"));
```

Output:

```
true
false
```

32. Safe `for...in` Pattern

যদি `for...in` ব্যবহার করো এবং শুধুমাত্র own properties process করতে চাও:

```
const user = {
  name: "Shariar",
  age: 31
};

for (const key in user) {
  if (Object.hasOwn(user, key)) {
    console.log(key, user[key]);
  }
}
```

এতে inherited enumerable properties skip করা হবে।

33. `for...in` VS `Object.keys()`

Object-এর own enumerable keys চাইলে:

```
Object.keys(user)
```

আর তারপর:

```
for (const key of Object.keys(user)) {  
  console.log(key, user[key]);  
}
```

এটা অনেক explicit।

34. `Object.entries()` Often Better

Key + value দুটোই দরকার হলে:

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
for (const [key, value] of Object.entries(user)) {  
  console.log(key, value);  
}
```

এটা খুব clean।

Mental model:

```
Object.keys()  
  ↓  
keys  
  
Object.values()  
  ↓  
values  
  
Object.entries()  
  ↓  
[key, value] pairs
```

35. Three Common Object Iteration Patterns

Keys

```
for (const key of Object.keys(user)) {  
  console.log(key);  
}
```

Values

```
for (const value of Object.values(user)) {  
  console.log(value);  
}
```

Keys + Values

```
for (const [key, value] of Object.entries(user)) {  
  console.log(key, value);  
}
```

36. `for...of` vs `for...in` — Core Difference

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

`for...in`

```
for (const key in user) {  
  console.log(key);  
}
```

Output:

```
name
  age
```

for...of

Plain object-এর উপর direct:

```
for (const value of user) {
  console.log(value);
}
```

✗ TypeError

But:

```
for (const value of Object.values(user)) {
  console.log(value);
}
```

Output:

```
Shariar
  31
```

37. Easy Memory Trick

এই দুইটা word আলাদা করে মনে রাখো:

```
OF
↓
WHAT IS INSIDE?

IN
↓
WHAT IS THE KEY/PROPERTY?
```

আরও সহজ:

```
for...of → value  
for...in → key
```

38. Array Example — Most Important

```
const numbers = [10, 20, 30];
```

for...of

```
for (const number of numbers) {  
  console.log(number);  
}
```

Output:

```
10  
20  
30
```

for...in

```
for (const index in numbers) {  
  console.log(index);  
}
```

Output:

```
0  
1  
2
```

39. Object Example — Most Important

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

for...in

```
for (const key in user) {  
  console.log(key);  
}
```

Output:

```
name  
age
```

Object + for...of

Directly:

```
for (const value of user) {  
  console.log(value);  
}
```

✗ Not valid for a normal plain object.

Instead:

```
for (const value of Object.values(user)) {  
  console.log(value);  
}
```

Output:

```
Shariar  
31
```

40. `for...of` with `break`

`for...of` -এ `break` ব্যবহার করা যায়।

```
const numbers = [10, 20, 30, 40, 50];

for (const number of numbers) {
  if (number === 30) {
    break;
  }

  console.log(number);
}
```

Output:

```
10
20
```

`break` loop পুরোপুরি stop করে।

41. `for...of` with `continue`

```
const numbers = [1, 2, 3, 4, 5];

for (const number of numbers) {
  if (number % 2 === 0) {
    continue;
  }

  console.log(number);
}
```

Output:

```
1
3
5
```

`continue` current iteration skip করে next iteration-এ যায়। `break` এবং `continue` নিয়ে বিস্তারিত **Doc 9.3**-তে আসবে।

42. `for...of` with Destructuring

Array of objects:

```
const users = [
  {
    name: "Shariar",
    age: 31
  },
  {
    name: "Duke",
    age: 30
  }
];
```

Instead of:

```
for (const user of users) {
  console.log(user.name);
}
```

You can do:

```
for (const { name, age } of users) {
  console.log(name, age);
}
```

Output:

```
Shariar 31
Duke 30
```

এটা modern JavaScript-এ খুব useful pattern।

43. `for...of` with Array Destructuring

```
const coordinates = [
  [10, 20],
  [30, 40],
  [50, 60]
];

for (const [x, y] of coordinates) {
  console.log(x, y);
}
```

Output:

```
10 20
30 40
50 60
```

44. `for...of` and Set

```
const tags = new Set([
  "javascript",
  "react",
  "nextjs"
]);

for (const tag of tags) {
  console.log(tag);
}
```

Output:

```
javascript
react
nextjs
```

45. `for...of` and Map

```
const scores = new Map([
  ["Shariar", 90],
  ["Duke", 85]
]);

for (const [name, score] of scores) {
  console.log(`${name}: ${score}`);
}
```

Output:

```
Shariar: 90
Duke: 85
```

46. Real-World API Example

API response:

```
const users = [
  {
    id: 1,
    name: "Shariar"
  },
  {
    id: 2,
    name: "Duke"
  }
];
```

Using `for...of`:

```
for (const user of users) {
  console.log(user.name);
}
```

Output:

Shariar

Duke

এখানে values নিয়ে কাজ করছি, তাই `for...of` natural choice।

47. Real-World Object Example

Suppose configuration:

```
const config = {  
  host: "localhost",  
  port: 3000,  
  environment: "development"  
};
```

Keys inspect করতে:

```
for (const key in config) {  
  console.log(key);  
}
```

Output:

```
host  
port  
environment
```

48. Form Data Example

```
const formData = {
  name: "Shariar",
  email: "example@email.com",
  age: 31
};

for (const [key, value] of Object.entries(formData)) {
  console.log(`${key}: ${value}`);
}
```

Output:

```
name: Shariar
email: example@email.com
age: 31
```

এই pattern dynamic form/API/configuration data-এর ক্ষেত্রে useful।

49. **for...of** Works with Strings, But Not Objects

```
for (const char of "Hello") {
  console.log(char);
}
```

✅ Works

```
for (const value of [1, 2, 3]) {
  console.log(value);
}
```

✅ Works

```
for (const value of new Set([1, 2, 3])) {  
  console.log(value);  
}
```

✅ Works

```
for (const value of { a: 1, b: 2 }) {  
  console.log(value);  
}
```

❌ Doesn't work by default

কারণ plain object iterable নয়।

50. Why `for...of` is Called "of"?

Think:

```
for (const value of array)
```

মানে:

"array-এর **values**-এর মধ্যে একটার পর একটা যাও।"

51. Why `for...in` is Called "in"?

Think:

```
for (const key in object)
```

মানে:

"object-এর **properties**-এর মধ্যে থাকা keys গুলো দেখাও।"

52. Important: `for...in` Doesn't Mean "Only

Objects"

Technically `for...in` arrays, strings, and other objects-এর enumerable properties-এর keys iterate করতে পারে।

তাই এই statement:

"`for...in` শুধু object-এর জন্য"

পুরোপুরি technically accurate না।

Better statement:

`for...in` property keys iterate করে এবং object properties inspect করার জন্য সবচেয়ে naturally useful।

53. Important: `for...of` Doesn't Mean "Only Arrays"

এটাও মনে রাখবে।

`for...of` arrays-এর জন্য popular, কিন্তু শুধু arrays-এর জন্য না।

Works with:

```
Array
String
Set
Map
TypedArray
other iterables
```

So:

`for...of` works with iterables.

54. `for...in` Property Order

Modern JavaScript-এ ordinary object-এর property enumeration order defined rules follow করে, তবে code logic-এর জন্য property order-এর ওপর unnecessary dependency avoid করাই ভালো।

যদি ordered collection হিসেবে data রাখতে চাও, তাহলে appropriate data structure যেমন Array বা Map বিবেচনা করতে পারো।

55. `for...in` + Prototype Pollution Concern

যদি object prototype chain থেকে inherited enumerable properties পেতে পারে, তাহলে blindly:

```
for (const key in object) {  
  // process  
}
```

করার আগে বুঝতে হবে তুমি own properties নাকি inherited properties—কোনটা process করতে চাচ্ছ।

Safer explicit approach:

```
for (const key in object) {  
  if (Object.hasOwn(object, key)) {  
    // process own property  
  }  
}
```

অথবা:

```
for (const [key, value] of Object.entries(object)) {  
  // own enumerable properties  
}
```

56. `Object.entries()` + `for...of`

Modern object iteration-এর জন্য খুব useful pattern:

```
const user = {
  name: "Shariar",
  age: 31,
  city: "Rangpur"
};

for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

এখানে তিনটি concept একসাথে কাজ করছে:

```
Object.entries()
    ↓
[key, value]
    ↓
for...of
    ↓
destructuring
```

57. **for...of** vs **forEach**

for...of:

```
for (const user of users) {
  console.log(user);
}
```

forEach:

```
users.forEach((user) => {
  console.log(user);
});
```

দুটোই values নিয়ে iterate করতে পারে।

কিন্তু **for...of** সাধারণ loop control statements যেমন:

```
break
  continue
```

support করে।

`forEach` callback-এর ভিতরে `break` / `continue` সরাসরি ব্যবহার করা যায় না।

58. `for...of` vs Traditional `for`

Traditional:

```
for (let i = 0; i < users.length; i++) {
  console.log(users[i]);
}
```

`for...of`:

```
for (const user of users) {
  console.log(user);
}
```

Traditional `for` সুবিধা

Index দরকার হলে:

```
users[i]
```

আর custom index manipulation দরকার হলে useful।

`for...of` সুবিধা

শুধু values দরকার হলে:

```
user
```

অনেক cleaner।

59. Choosing the Right Loop

Array values

```
for...of
```

Array index + value

```
for
```

or:

```
for...of + entries()
```

Object keys

```
for...in
```

or:

```
Object.keys() + for...of
```

Object values

```
Object.values() + for...of
```

Object key + value

```
Object.entries() + for...of
```

String characters

```
for...of
```

Set values

```
for...of
```

Map entries

```
for...of
```

60. Cheat Sheet

ARRAY

↓

for...of → values

for...in → indexes/keys (generally avoid)

OBJECT

↓

for...in → keys

Object.values() + for...of → values

Object.entries() + for...of → key + value

STRING

↓

for...of → characters

SET

↓

for...of → values

MAP

↓

for...of → [key, value]

61. Practice — Output Prediction

Question 1

```
const numbers = [10, 20, 30];

for (const value of numbers) {
  console.log(value);
}
```

Output कै?

Question 2

```
const numbers = [10, 20, 30];

for (const key in numbers) {
  console.log(key);
}
```

Question 3

```
const user = {
  name: "Shariar",
  age: 31
};

for (const key in user) {
  console.log(key);
}
```

Question 4

```
const user = {  
  name: "Shariar",  
  age: 31  
};  
  
for (const key in user) {  
  console.log(user[key]);  
}
```

Question 5

```
const text = "ABC";  
  
for (const char of text) {  
  console.log(char);  
}
```

Question 6

```
const set = new Set([1, 1, 2, 2, 3]);  
  
for (const value of set) {  
  console.log(value);  
}
```

Question 7

```
const user = {
  name: "Shariar",
  age: 31
};

for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

62. Practice — Find the Mistake

Problem 1

```
const numbers = [10, 20, 30];

for (const number of numbers) {
  console.log(numbers[number]);
}
```

এখানে problem কী?

Hint:

`number` কী represent করছে?

Problem 2

```
const user = {
  name: "Shariar",
  age: 31
};

for (const value of user) {
  console.log(value);
}
```

কেন error হবে?

Problem 3

```
const numbers = [10, 20, 30];

for (const key in numbers) {
  console.log(numbers[key]);
}
```

এটা technically কাজ করবে। কিন্তু array values iterate করার জন্য কেন `for...of` বেশি appropriate?

63. Coding Challenge 1 — Array Values

Given:

```
const numbers = [10, 20, 30, 40, 50];
```

`for...of` ব্যবহার করে:

- প্রতিটি number print করো
- total sum বের করো

Expected sum:

```
150
```

64. Coding Challenge 2 — String Characters

Given:

```
const text = "javascript";
```

`for...of` ব্যবহার করে:

- প্রতিটি character print করো
- vowel count বের করো

Expected:

```
3
```

65. Coding Challenge 3 — Object Keys

Given:

```
const user = {  
  name: "Shariar",  
  age: 31,  
  city: "Rangpur",  
  role: "Developer"  
};
```

`for...in` ব্যবহার করে প্রতিটি key print করো।

Expected:

```
name  
age  
city  
role
```

66. Coding Challenge 4 — Object Key + Value

Given:

```
const product = {  
  name: "Laptop",  
  price: 85000,  
  brand: "HP"  
};
```

Output:

```
name: Laptop  
price: 85000  
brand: HP
```

Hint:

```
Object.entries()
```

এর সাথে `for...of` ব্যবহার করো।

67. Coding Challenge 5 — Find User

Given:

```
const users = [  
  {  
    id: 1,  
    name: "Shariar"  
  },  
  {  
    id: 2,  
    name: "Duke"  
  },  
  {  
    id: 3,  
    name: "Rahim"  
  }  
];
```

`for...of` ব্যবহার করে `id === 2` user খুঁজে বের করার চেষ্টা করো।
এখানে `later` শেখা `break` ব্যবহার করতে পারো।

68. Coding Challenge 6 — Set

Given:

```
const numbers = new Set([  
  10,  
  20,  
  20,  
  30,  
  30,  
  40  
]);
```

`for...of` ব্যবহার করে সব unique value print করো।

Expected:

```
10
20
30
40
```

69. Coding Challenge 7 — Map

Given:

```
const scores = new Map([
  ["Shariar", 90],
  ["Duke", 85],
  ["Rahim", 88]
]);
```

`for...of` + destructuring ব্যবহার করে output করো:

```
Shariar scored 90
Duke scored 85
Rahim scored 88
```

70. Interview Questions

Beginner

1. What is `for...of`?
 2. What does `for...of` return during iteration?
 3. What is `for...in`?
 4. What does `for...in` iterate?
 5. What's the main difference between `for...of` and `for...in`?
 6. Can `for...of` be used with arrays?
 7. Can `for...of` be used with strings?
 8. Can `for...of` be used with objects?
-

Intermediate

9. Why is `for...of` generally preferred over `for...in` for array values?
 10. What is an iterable?
 11. Name some built-in iterables.
 12. Why doesn't `for...of` work directly with a plain object?
 13. How can you use `for...of` with an object?
 14. How do you get both key and value from an object?
 15. What is the difference between `Object.keys()` and `Object.entries()`?
 16. How can inherited properties affect `for...in`?
 17. What is `Object.hasOwn()`?
-

Advanced

18. Explain the difference between iterable and enumerable.
 19. Why can `for...in` be problematic for arrays?
 20. How does `Object.entries()` + `for...of` compare with `for...in`?
 21. What happens if an object has enumerable properties on its prototype?
 22. How does destructuring work with `for...of`?
 23. Why does `for...of` work with Map?
 24. What does a Map's default iterator return?
 25. Why is `for...of` useful when you need `break` / `continue`?
-

71. Quick Revision Table

Situation	Recommended
Array values	<code>for...of</code>
Array index	traditional <code>for</code>
Array index + value	<code>entries()</code> + <code>for...of</code>
Object keys	<code>for...in</code> / <code>Object.keys()</code>
Object values	<code>Object.values()</code> + <code>for...of</code>
Object key + value	<code>Object.entries()</code> + <code>for...of</code>
String characters	<code>for...of</code>
Set values	<code>for...of</code>
Map entries	<code>for...of</code>
Need <code>break</code> / <code>continue</code>	<code>for</code> / <code>for...of</code>

72. The Most Important Comparison

for...of

Gives VALUES

Array → 10, 20, 30

String → A, B, C

Set → values

Map → [key, value]

Other iterables

for...in

Gives KEYS

Object → property names

Array → indexes

Includes enumerable

inherited properties

73. Golden Rules

Rule 1

`for...of` → *values*

Rule 2

`for...in` → *keys/property names*

Rule 3

Array values iterate করতে generally `for...of` use করো।

Rule 4

Plain object direct `for...of` করা যায় না।

Rule 5

Object values চাইলে:

```
Object.values(object)
```

এর সাথে:

```
for...of
```

use করতে পারো।

Rule 6

Object key + value চাইলে:

```
Object.entries(object)
```

এর সাথে:

```
for...of
```

খুব clean pattern।

Rule 7

`for...in` ব্যবহার করলে inherited enumerable properties-এর possibility মনে রাখবে।

Rule 8

Own properties নিশ্চিত করতে:

```
Object.hasOwn(object, key)
```

ব্যবহার করতে পারো।

Rule 9

`for...of` শুধু `Array`-এর জন্য নয়—যেকোনো `iterable`-এর জন্য।

Rule 10

এই `line`টা `permanently` মনে রাখো:

```
for...of → VALUE  
for...in → KEY
```



Final Mental Model

ধরো:

```
const numbers = [10, 20, 30];
```

তখন:

```
for (const value of numbers) {  
  // value → 10, 20, 30  
}
```

মানে:

`Array`-এর ভিতরের জিনিসগুলো আমাকে দাও।

আর:

```
for (const key in numbers) {  
  // key → "0", "1", "2"  
}
```

মানে:

Array/object-এর property keys আমাকে দাও।

আর object:

```
const user = {  
  name: "Shariar",  
  age: 31  
};
```

এখানে:

```
for (const key in user) {  
  // key → "name", "age"  
}
```

আর values চাইলে:

```
for (const value of Object.values(user)) {  
  // value → "Shariar", 31  
}
```

আর key + value চাইলে:

```
for (const [key, value] of Object.entries(user)) {  
  // key + value  
}
```

এই চারটি pattern ভালোভাবে আয়ত্ত করলেই object/array iteration-এর বড় একটা অংশ clear হয়ে যাবে।



Doc 9.2 Mastery Checklist

- ☐ `for...of`
- ☐ `for...in`
- ☐ `for...of` values দেয়
- ☐ `for...in` keys দেয়
- ☐ Array + `for...of`
- ☐ Array + `for...in`
- ☐ কেন array-এর জন্য `for...in` avoid করা হয়
- ☐ String + `for...of`

- ☐ Set + `for...of`
- ☐ Map + `for...of`
- ☐ Map keys
- ☐ Map values
- ☐ Map entries
- ☐ Iterable concept
- ☐ Plain object কেন `for...of` support করে না
- ☐ `Object.keys()`
- ☐ `Object.values()`
- ☐ `Object.entries()`
- ☐ Object + `for...in`
- ☐ Own property
- ☐ Inherited property
- ☐ `Object.hasOwn()`
- ☐ Destructuring with `for...of`
- ☐ `break` / `continue` with `for...of`
- ☐ `for...of` vs `forEach`
- ☐ `for...of` vs traditional `for`



Next — Doc 9.3

শেষ doc-এ আসবে:

Doc 9.3 – break, continue & Labeled Statements

- └─ break
- └─ continue
- └─ Loop early exit
- └─ Skip iteration
- └─ Nested loops
- └─ break in switch
- └─ Labeled statements
- └─ Labeled break
- └─ Labeled continue
- └─ Practical + DSA patterns

এখানে সবচেয়ে important mental model হবে:

break

↓

পুরো loop STOP

continue

↓

current iteration SKIP

↓

next iteration

labeled break

↓

specific outer loop EXIT